PRACTICAL-4

AIM: Add dynamic UI components such as collapsible FAQ, modal popup, image/content slider, notification banner, hamburger menu, and light/dark theme switcher using JavaScript.

SOURCE CODE :

https://github.com/rachhadiyabrind-del/WDF_B/tree/main/Practical(1-4)(Studenthub)/javascript

PREVIEW:

KEY QUESTIONS :

1. How are DOM elements selected and modified?

DOM elements are selected using JavaScript methods such as:

- document.getElementById() — selects an element by its ID.
- document.querySelector() — selects the first matching element.
- document.querySelectorAll() — selects multiple matching elements.

For example, in the project:

const hamburgerBtn = document.getElementById('hamburger-btn');

const navElement = document.querySelector('header nav');

The selected elements are modified using:

- classList.add()
- classList.remove()
- classList.toggle()

- setAttribute()
- textContent

For example, when the hamburger menu is clicked:

const isOpen = navElement.classList.toggle('nav-active');

hamburgerBtn.setAttribute('aria-expanded', isOpen ? 'true' : 'false');

The project also dynamically changes the greeting using textContent and changes the theme by adding/removing the dark-theme class.

Answer: DOM elements are selected using getElementById(), querySelector(), and querySelectorAll(), and modified using classList, setAttribute(), and textContent.

2. Are event listeners attached correctly?

Yes. Event listeners are attached using addEventListener() for different interactive elements.

Examples from the project include:

- click → hamburger menu, theme button, FAQ, modal, notification
- keydown → closing the modal with the Escape key
- change → theme selection in settings
- resize → handling the responsive navigation
- DOMContentLoaded → initializing the JavaScript functionality after the page loads

For example:

```
hamburgerBtn.addEventListener('click', function () {
  // toggle navigation
});
```

The FAQ also uses:

```
button.addEventListener('click', function () {
  // open/close FAQ
});
```

The modal uses a keyboard event:

```
document.addEventListener('keydown', function (event) {
  if (event.key === 'Escape') {
    // close modal
  }
});
```

Answer: Yes, event listeners are correctly attached to the required interactive elements using addEventListener(), allowing the UI components to respond to user actions.

3. Is localStorage used for remembering theme choice?

Yes. Your project uses localStorage specifically for remembering the user's light/dark theme choice.

The selected theme is stored using:

localStorage.setItem(THEME_STORAGE_KEY, newTheme);

When the page loads, the saved theme is retrieved:

const savedTheme = localStorage.getItem(THEME_STORAGE_KEY) || 'light';

Then it is applied to the page:

applyTheme(savedTheme);

So, if the user selects Dark Mode, leaves the page, and opens another StudentHub page, the saved theme can be restored.

Answer: Yes. localStorage stores the selected light/dark theme and retrieves it when the page loads, so the user's theme preference is remembered.

4. Does interactivity improve usability without breaking accessibility?

Yes. The project uses JavaScript interactivity while also including several accessibility features.

Examples:

- The hamburger menu updates aria-expanded:

```
hamburgerBtn.setAttribute('aria-expanded', isOpen ? 'true' : 'false');
```

- The theme button has an accessible label:

aria-label="Toggle dark/light theme"

- The navigation has:

<nav aria-label="Primary Navigation">

- Images have meaningful alt text, such as:

alt="StudentHub Logo"

- The FAQ uses aria-expanded and updates it when the FAQ opens/closes.
- The modal uses role="dialog", aria-modal, and aria-hidden.
- The modal can also be closed using the Escape key.
- The modal moves focus to an interactive element when opened.
- Form fields use associated <label> elements.

Therefore, the JavaScript features improve usability while the ARIA attributes, keyboard interaction, labels, and focus handling help maintain accessibility.

Answer: Yes. Interactivity improves usability through menus, FAQ, modals, notifications, theme switching, etc., while accessibility is maintained using ARIA attributes, labels, keyboard support, focus management, and meaningful alt text.